Emojicoin

Security Assessment

February 17th, 2025 — Prepared by OtterSec

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-EMJ-ADV-00 | Frontrunning Matched Funds for Unfair Gains 6

General Findings **7**

OS-EMJ-SUG-00 | Code Maturity 8

Appendices

Vulnerability Rating Scale **9**

Procedure **10**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Econia Labs engaged OtterSec to assess the `emojicoin-dot-fun` program. This assessment was conducted between February 7th and February 12th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 2 findings throughout this audit engagement.

In particular, the emojicoin arena module is vulnerable to frontrunning by creating multiple pools and strategically buying tokens to exploit matched funds, potentially resulting in unfair gains (OS-EMJ-ADV-00).

We also made suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices (OS-EMJ-SUG-00).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/econia-labs/emojicoin-dot-fun. This audit was performed against commit 63de59e.

A brief description of the program is as follows:

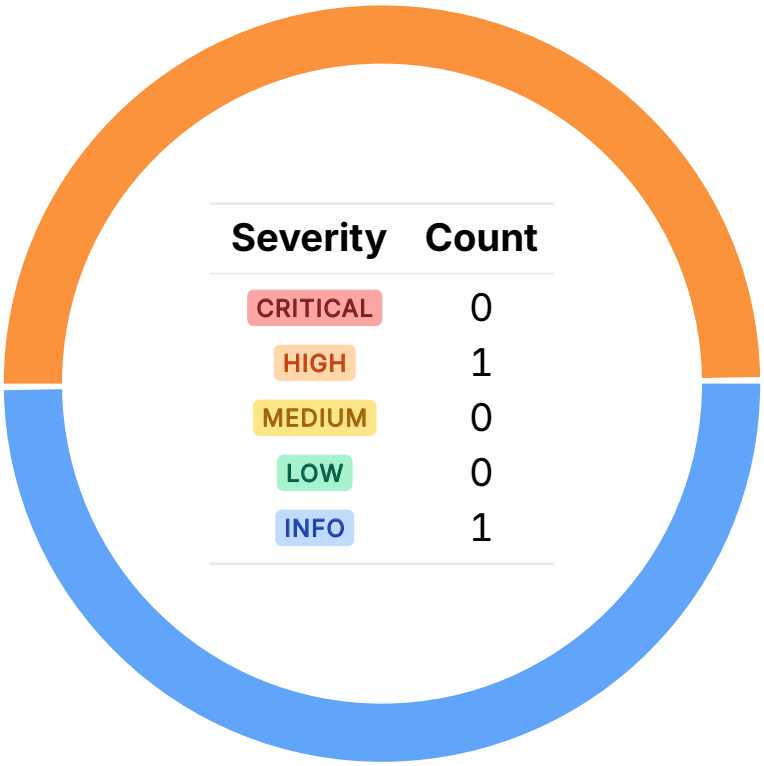
Name	Description
emojicoin-dot-fun	A gamified trading platform where users compete by trading between two randomly chosen emojicoins during timed events called melees. Participants swap APT for emojicoins held in escrow, enforcing all-or-nothing trading rules. Rewards are distributed from a vault, with early commitment earning higher matching bonuses. Profit and loss are tracked via leaderboards, adding a competitive edge to the trading experience.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 2 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	1
MEDIUM	0
LOW	0
INFO	1

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-EMJ-ADV-00	HIGH	TODO	The emojicoin arena module is vulnerable to frontrunning through the creation of multiple pools and the strategic purchase of tokens to exploit matched funds, potentially resulting in unfair gains.

© 2025 Otter Audits LLC. All Rights Reserved.

Frontrunning Matched Funds for Unfair Gains HIGH OS-EMJ-ADV-00

Description

There is potential for frontrunning when matching funds are allocated. This issue arises due to the way matched amounts are distributed. The emojicoin arena module features a mechanism where users may lock in a portion of their contribution to receive matched funds from the vault. An attacker may create a large number of pools with small amounts, increasing the likelihood that one of their pools is chosen during the crank scheduling.

Before the crank selects a melee, the attacker may buy a large amount of their own token, driving up its price, inflating its value relative to other tokens in the pool. Consequently, if their pool is selected, they may then buy into the pool and swap out their tokens to capture the matched funds.

Remediation

Limit the number of pools a single address may create to prevent spamming the crank with attacker's pools.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-EMJ-SUG-00	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity

OS-EMJ-SUG-00

Description

1. The utilization of the modulo operator (`%`) in `u64_range` introduces a bias that renders the output non-uniformly random, which may create predictable patterns. When utilizing `% range` to map a large number space (`u256`) to a smaller one (`u64` range), the distribution becomes uneven if the range does not evenly divide the maximum possible value of `u256`. This occurs because some numbers in the `u256` space will wrap around and have a slightly higher chance of occurring compared to others. Ensure there is no unintended bias in the random generation process.

   >_ emojicoin_arena/sources/pseudo_randomness.move RUST

   ```rust
   /// Pseudo-random substitute for `aptos_framework::randomness::u64_range`, since
   /// the randomness API is not available during `init_module`.
   public(friend) inline fun u64_range(min_incl: u64, max_excl: u64): u64 {
       let range = ((max_excl - min_incl) as u256);
       let sample = ((u256_integer() % range) as u64);
       min_incl + sample
   }
   ```

2. In the current implementation, type checking is delayed until `emojicoin_dot_fun::swap`, allowing multiple state modifications to occur beforehand. This exposes the contract to potential inconsistencies due to type confusion. If an invalid type is detected at this late stage, the transaction may fail after partial state changes, violating best practices and impacting the program's overall functionality. To prevent this, type validation should be performed as early as possible.

Remediation

Implement the above-mentioned suggestions.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.